Python: Recap II

This notebook was generated in **solutions** mode.

Exercise 1: Circle

Exercise Description

Create a Python class called `Circle` that models a geometric circle. The class should have the following features:

1. **Attributes**:
    - `radius`: A numeric attribute representing the radius of the circle.
2. **Methods**:
    - `__init__(self, radius)`: A constructor method to initialize the `radius` attribute when a `Circle` object is created.
    - `get_area(self)`: A method that returns the area of the circle (use the formula $\pi \times \text{radius}^2$).
    - `get_perimeter(self)`: A method that returns the perimeter (circumference) of the circle (use the formula $2 \times \pi \times \text{radius}$).
    - `display_info(self)`: A method that prints the radius, area, and perimeter of the circle.

Solution

Test your code

Run this code to test your solution:

Your solution

```
import math

class Circle:
```

```
    def __init__(self, radius):
        self.radius = radius

    def get_area(self):
        return math.pi * (self.radius ** 2)

    def get_perimeter(self):
        return 2 * math.pi * self.radius

    def display_info(self):
        print(f"Circle with radius: {self.radius}")
        print(f"Area: {self.get_area():.2f}")
        print(f"Perimeter: {self.get_perimeter():.2f}")
```

Test your solution

```
# Test the Circle class
circle = Circle(5)
assert circle.radius == 5, "Test failed: Incorrect radius"

# Test area calculation
area = circle.get_area()
assert abs(area - 78.54) < 0.1, "Test failed: Incorrect area calculation"

# Test perimeter calculation
perimeter = circle.get_perimeter()
assert abs(perimeter - 31.42) < 0.1, "Test failed: Incorrect perimeter calculation"

# Test display_info method exists
circle.display_info()
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Library Book

Exercise Description

Create a Python class called `Book` that models a book in a library. The class should have the following features:

1. **Attributes**:

    - `title`: A string representing the title of the book.
    - `author`: A string representing the author of the book.
    - `isbn`: A string representing the unique ISBN number of the book.
    - `is_checked_out`: A boolean attribute indicating if the book is currently checked out (default is `False`).

2. **Methods**:

    - `__init__(self, title, author, isbn)`: A constructor method that initializes the title, author, isbn, and sets `is_checked_out` to `False`.
    - `check_out(self)`: Marks the book as checked out by setting `is_checked_out` to `True` and returns `True`. If the book is already checked out, returns `False`.
    - `return_book(self)`: Marks the book as returned by setting `is_checked_out` to `False` and returns `True`. If the book is not checked out, returns `False`.
    - `get_info(self)`: returns, as a string, the details of the book, including its title, author, ISBN, and availability status.

Solution

Test your code

Run this code to test your solution:

Your solution

```
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.is_checked_out = False

    def check_out(self):
        if not self.is_checked_out:
            self.is_checked_out = True
            return True
        else:
            return False

    def return_book(self):
        if self.is_checked_out:
            self.is_checked_out = False
            return True
        else:
            return False

    def get_info(self):
        status = "Available" if not self.is_checked_out else "Checked out"
        return f"Title: {self.title}\nAuthor: {self.author}\nISBN: {self.isbn}\nStatu
```

Test your solution

```python
# Test the Book class
book = Book("1984", "George Orwell", "123-4567890123")
assert book.title == "1984", "Test failed: Incorrect title"
assert book.author == "George Orwell", "Test failed: Incorrect author"
assert book.isbn == "123-4567890123", "Test failed: Incorrect ISBN"
assert book.is_checked_out == False, "Test failed: Book should be initially available

# Test check out
result = book.check_out()
assert result == True, "Test failed: Book should be checked out successfully"
assert book.is_checked_out == True, "Test failed: Book should be marked as checked ou

# Test return
result = book.return_book()
assert result == True, "Test failed: Book should be returned successfully"
assert book.is_checked_out == False, "Test failed: Book should be marked as available
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Shapes

Exercise Description

Create a base class called `Shape` with two derived classes, `Rectangle` and `Circle`, each with specific attributes. Override the `__eq__` operator in the base class to allow comparison between instances based on their attributes.

Class 1: `Shape` (Base Class)

- **Attributes**:
  - `name`: A string representing the name of the shape.
- **Methods**:
  - `__init__(self, name)`: Initializes the `name` attribute.
  - `__eq__(self, other)`: Returns `True` if the `name` and specific attributes of two shapes are identical, and `False` otherwise.
  - `area(self)`: A placeholder method that should be overridden by subclasses to calculate the area. If called directly, it should generate an error using `raise NotImplementedError`.

Class 2: `Rectangle` (inherits from `Shape`)

- **Additional Attributes**:
    - `width`: The width of the rectangle (a float).
    - `height`: The height of the rectangle (a float).
- **Methods**:
    - `__init__(self, width, height)`: Initializes the `width` and `height` attributes and sets `name` to "Rectangle".
    - `area(self)`: Returns the area of the rectangle (`width * height`).
    - `__eq__(self, other)`: Checks if two rectangles are equal by comparing `width` and `height` values in addition to `name`.

Class 3: `Circle` (inherits from `Shape`)

- **Additional Attributes**:
    - `radius`: The radius of the circle (a float).
- **Methods**:
    - `__init__(self, radius)`: Initializes the `radius` attribute and sets `name` to "Circle".
    - `area(self)`: Returns the area of the circle (`π * radius^2`).
    - `__eq__(self, other)`: Checks if two circles are equal by comparing their `radius` values in addition to `name`.

Your solution

```python
# Base Class
class Shape:
    def __init__(self, name):
        self.name = name

    def __eq__(self, other):
        if not isinstance(other, Shape):
            return False
        return self.name == other.name

    def area(self):
        raise NotImplementedError("Subclasses should implement the area method")

# Derived Class: Rectangle
class Rectangle(Shape):
    def __init__(self, width, height):
        super().__init__("Rectangle")
        self.width = float(width)
        self.height = float(height)

    def area(self):
        return self.width * self.height

    def __eq__(self, other):
        if not isinstance(other, Rectangle):
            return False
        return super().__eq__(other) and self.width == other.width and self.height ==

    def __str__(self):
        return f"{self.name} (Width: {self.width}, Height: {self.height})"
```

```python
# Derived Class: Circle
class Circle(Shape):
    def __init__(self, radius):
        super().__init__("Circle")
        self.radius = float(radius)

    def area(self):
        return math.pi * (self.radius ** 2)

    def __eq__(self, other):
        if not isinstance(other, Circle):
            return False
        return super().__eq__(other) and self.radius == other.radius

    def __str__(self):
        return f"{self.name} (Radius: {self.radius})"
```

Test your solution

```python
# Test Rectangle initialization
rect = Rectangle(3, 4)
assert rect.width == 3.0, "Rectangle width initialization failed"
assert rect.height == 4.0, "Rectangle height initialization failed"
assert rect.name == "Rectangle", "Rectangle name initialization failed"

# Test Circle initialization
circle = Circle(5)
assert circle.radius == 5.0, "Circle radius initialization failed"
assert circle.name == "Circle", "Circle name initialization failed"

import math
```

```python
# Test area calculation for Rectangle
rect = Rectangle(3, 4)
assert rect.area() == 12.0, "Rectangle area calculation failed"

# Test area calculation for Circle
circle = Circle(5)
expected_area = math.pi * (5 ** 2)
assert math.isclose(circle.area(), expected_area, rel_tol=1e-9), "Circle area calcula

# Test basic equality between two shapes of the same type
rect1 = Rectangle(3, 4)
rect2 = Rectangle(3, 4)
rect3 = Rectangle(5, 6)

assert rect1 == rect2, "Rectangles with the same dimensions should be equal"
assert rect1 != rect3, "Rectangles with different dimensions should not be equal"

# Test equality for circles
circle1 = Circle(5)
circle2 = Circle(5)
circle3 = Circle(10)

assert circle1 == circle2, "Circles with the same radius should be equal"
assert circle1 != circle3, "Circles with different radii should not be equal"

# Test inequality between different shape types
assert rect1 != circle1, "Rectangle and Circle should not be equal"
```

Test your solution

```python
# Test Rectangle initialization
rect = Rectangle(3, 4)
assert rect.width == 3.0, "Rectangle width initialization failed"
assert rect.height == 4.0, "Rectangle height initialization failed"
assert rect.name == "Rectangle", "Rectangle name initialization failed"

# Test Circle initialization
circle = Circle(5)
assert circle.radius == 5.0, "Circle radius initialization failed"
assert circle.name == "Circle", "Circle name initialization failed"

import math

# Test area calculation for Rectangle
rect = Rectangle(3, 4)
assert rect.area() == 12.0, "Rectangle area calculation failed"

# Test area calculation for Circle
circle = Circle(5)
expected_area = math.pi * (5 ** 2)
assert math.isclose(circle.area(), expected_area, rel_tol=1e-9), "Circle area calcula

# Test basic equality between two shapes of the same type
rect1 = Rectangle(3, 4)
rect2 = Rectangle(3, 4)
rect3 = Rectangle(5, 6)

assert rect1 == rect2, "Rectangles with the same dimensions should be equal"
assert rect1 != rect3, "Rectangles with different dimensions should not be equal"

# Test equality for circles
```

```
circle1 = Circle(5)
circle2 = Circle(5)
circle3 = Circle(10)

assert circle1 == circle2, "Circles with the same radius should be equal"
assert circle1 != circle3, "Circles with different radii should not be equal"

# Test inequality between different shape types
assert rect1 != circle1, "Rectangle and Circle should not be equal"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Vectors

Exercise Description

Create two Python classes, `Vector2D` and `Vector3D`, to represent vectors in 2D and 3D space. The `Vector3D` class should inherit from `Vector2D`, adding functionality specific to 3D vectors. Override arithmetic operators to perform vector operations in both classes.

Class 1: `Vector2D`

- **Attributes**:
  - `x`: The x-coordinate of the vector (a float).
  - `y`: The y-coordinate of the vector (a float).
- **Methods**:
  - `__init__(self, x, y)`: Initializes the x and y attributes.
  - `__add__(self, other)`: Defines vector addition (`+` operator).
  - `__sub__(self, other)`: Defines vector subtraction (`-` operator).
  - `__mul__(self, scalar)`: Defines scalar multiplication (`*` operator) where `scalar` is a float or an int.
  - `__str__(self)`: Returns a string representation of the vector such as `"(x, y)"`.

Class 2: `Vector3D` (inherits from `Vector2D`)

- **Additional Attributes**:
    - `z`: The z-coordinate of the vector (a float).
- **Methods**:
    - `__init__(self, x, y, z)`: Initializes x, y (from `Vector2D`), and z attributes.
    - `__add__(self, other)`: Overrides addition to handle 3D vector addition.
    - `__sub__(self, other)`: Overrides subtraction to handle 3D vector subtraction.
    - `__mul__(self, scalar)`: Overrides multiplication to handle 3D vector multiplication.
    - `__str__(self)`: Returns a string representation of the 3D vector such as `"(x, y, z)"`.

Your solution

```
class Vector2D:
    def __init__(self, x, y):
        self.x = float(x)
        self.y = float(y)

    def __add__(self, other):
        if isinstance(other, Vector2D):
            return Vector2D(self.x + other.x, self.y + other.y)
        raise ValueError("Addition is only supported between Vector2D instances.")

    def __sub__(self, other):
        if isinstance(other, Vector2D):
            return Vector2D(self.x - other.x, self.y - other.y)
```

```
        raise ValueError("Subtraction is only supported between Vector2D instances.")

    def __mul__(self, scalar):
        if isinstance(scalar, (int, float)):
            return Vector2D(self.x * scalar, self.y * scalar)
        raise ValueError("Multiplication is only supported with a scalar.")

    def __str__(self):
        return f"({self.x}, {self.y})"

class Vector3D(Vector2D):
    def __init__(self, x, y, z):
        super().__init__(x, y)
        self.z = float(z)

    def __add__(self, other):
        if isinstance(other, Vector3D):
            return Vector3D(self.x + other.x, self.y + other.y, self.z + other.z)
        raise ValueError("Addition is only supported between Vector3D instances.")

    def __sub__(self, other):
        if isinstance(other, Vector3D):
            return Vector3D(self.x - other.x, self.y - other.y, self.z - other.z)
        raise ValueError("Subtraction is only supported between Vector3D instances.")

    def __mul__(self, scalar):
        if isinstance(scalar, (int, float)):
            return Vector3D(self.x * scalar, self.y * scalar, self.z * scalar)
        raise ValueError("Multiplication is only supported with a scalar.")
```

```
    def __str__(self):
        return f"({self.x}, {self.y}, {self.z})"
```

Test your solution

```
# Test initialization
v2d = Vector2D(3, 4)
assert v2d.x == 3.0, "Vector2D initialization failed for x"
assert v2d.y == 4.0, "Vector2D initialization failed for y"

# Test addition
v2d_1 = Vector2D(1, 2)
v2d_2 = Vector2D(3, 4)
v2d_add = v2d_1 + v2d_2
assert v2d_add.x == 4.0 and v2d_add.y == 6.0, "Vector2D addition failed"

# Test subtraction
v2d_sub = v2d_2 - v2d_1
assert v2d_sub.x == 2.0 and v2d_sub.y == 2.0, "Vector2D subtraction failed"

# Test scalar multiplication
v2d_mul = v2d_1 * 2
assert v2d_mul.x == 2.0 and v2d_mul.y == 4.0, "Vector2D scalar multiplication failed"

# Test string representation
assert str(v2d) == "(3.0, 4.0)", "Vector2D string representation failed"

# Test initialization
v3d = Vector3D(1, 2, 3)
assert v3d.x == 1.0, "Vector3D initialization failed for x"
assert v3d.y == 2.0, "Vector3D initialization failed for y"
```

```
assert v3d.z == 3.0, "Vector3D initialization failed for z"

# Test addition
v3d_1 = Vector3D(1, 2, 3)
v3d_2 = Vector3D(4, 5, 6)
v3d_add = v3d_1 + v3d_2
assert (v3d_add.x, v3d_add.y, v3d_add.z) == (5.0, 7.0, 9.0), "Vector3D addition faile

# Test subtraction
v3d_sub = v3d_2 - v3d_1
assert (v3d_sub.x, v3d_sub.y, v3d_sub.z) == (3.0, 3.0, 3.0), "Vector3D subtraction fa

# Test scalar multiplication
v3d_mul = v3d_1 * 2
assert (v3d_mul.x, v3d_mul.y, v3d_mul.z) == (2.0, 4.0, 6.0), "Vector3D scalar multipl

# Test string representation
assert str(v3d) == "(1.0, 2.0, 3.0)", "Vector3D string representation failed"
```

Test your solution

```
# Test initialization
v2d = Vector2D(3, 4)
assert v2d.x == 3.0, "Vector2D initialization failed for x"
assert v2d.y == 4.0, "Vector2D initialization failed for y"

# Test addition
v2d_1 = Vector2D(1, 2)
v2d_2 = Vector2D(3, 4)
v2d_add = v2d_1 + v2d_2
assert v2d_add.x == 4.0 and v2d_add.y == 6.0, "Vector2D addition failed"
```

```python
# Test subtraction
v2d_sub = v2d_2 - v2d_1
assert v2d_sub.x == 2.0 and v2d_sub.y == 2.0, "Vector2D subtraction failed"

# Test scalar multiplication
v2d_mul = v2d_1 * 2
assert v2d_mul.x == 2.0 and v2d_mul.y == 4.0, "Vector2D scalar multiplication failed"

# Test string representation
assert str(v2d) == "(3.0, 4.0)", "Vector2D string representation failed"

# Test initialization
v3d = Vector3D(1, 2, 3)
assert v3d.x == 1.0, "Vector3D initialization failed for x"
assert v3d.y == 2.0, "Vector3D initialization failed for y"
assert v3d.z == 3.0, "Vector3D initialization failed for z"

# Test addition
v3d_1 = Vector3D(1, 2, 3)
v3d_2 = Vector3D(4, 5, 6)
v3d_add = v3d_1 + v3d_2
assert (v3d_add.x, v3d_add.y, v3d_add.z) == (5.0, 7.0, 9.0), "Vector3D addition faile

# Test subtraction
v3d_sub = v3d_2 - v3d_1
assert (v3d_sub.x, v3d_sub.y, v3d_sub.z) == (3.0, 3.0, 3.0), "Vector3D subtraction fa

# Test scalar multiplication
v3d_mul = v3d_1 * 2
assert (v3d_mul.x, v3d_mul.y, v3d_mul.z) == (2.0, 4.0, 6.0), "Vector3D scalar multipl
```

```
# Test string representation
assert str(v3d) == "(1.0, 2.0, 3.0)", "Vector3D string representation failed"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Books and Library

Exercise Description

Create two Python classes, `Library` and `Book`, to represent a system where a library can manage a collection of books. The classes should have the following features:

Class 1: `Book`

1. **Attributes**:

    - `title`: A string representing the title of the book.
    - `author`: A string representing the author of the book.
    - `isbn`: A string representing the unique ISBN number of the book.
    - `is_checked_out`: A boolean attribute indicating if the book is currently checked out (default is `False`).

2. **Methods**:

    - `__init__(self, title, author, isbn)`: Initializes the `title`, `author`, `isbn`, and sets `is_checked_out` to `False`.
    - `check_out(self)`: Marks the book as checked out by setting `is_checked_out` to `True` and returns `True`. If the book is already checked out, returns `False`.
    - `return_book(self)`: Marks the book as returned by setting `is_checked_out` to `False` and returns `True`. If the book is not checked out, returns `False`.
    - `get_info(self)`: returns, as a string, the details of the book, including its title, author, ISBN, and availability status.
    - `get_title(self)`: returns, as a string, the title of the book.

Class 2: `Library`

1. **Attributes**:

    - `name`: A string representing the name of the library.
    - `books`: A list that holds instances of `Book` objects.

2. **Methods**:

    - `__init__(self, name)`: Initializes the library with a name and an empty list of books.
    - `add_book(self, book)`: Adds a `Book` instance to the library's collection and returns `True`. If the book is already in the library, returns `False`.
    - `list_books(self)`: returns a list of the titles of all books in the library.
    - `list_available_books(self)`: returns a list of the titles of all available books in the library.
    - `check_out_book(self, isbn)`: Checks out a book by its ISBN if it is available, returning `True`. If the book is not available or does not exist, returns `False`.
    - `return_book(self, isbn)`: Returns a book by its ISBN if it is checked out, returning `True`. If the book is not checked out or does not exist, returns `False`.

Solution

Test your code

Run this code to test your solution:

Your solution

```python
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.is_checked_out = False

    def check_out(self):
        if not self.is_checked_out:
            self.is_checked_out = True
            return True
        else:
            return False

    def return_book(self):
        if self.is_checked_out:
            self.is_checked_out = False
            return True
        else:
            return False

    def get_info(self):
        status = "Available" if not self.is_checked_out else "Checked out"
        return f"Title: {self.title}\nAuthor: {self.author}\nISBN: {self.isbn}\nStatu

    def get_title(self):
        return self.title
```

```
class Library:
    def __init__(self, name):
        self.name = name
        self.books = []

    def add_book(self, book):
        if not isinstance(book, Book):
            return False
        if book not in self.books:
            self.books.append(book)
            return True
        else:
            return False

    def list_books(self):
        books = []
        for book in self.books:
            books.append(book.get_title())
        return books

    def list_available_books(self):
        books = []
        for book in self.books:
            if not book.is_checked_out:
                books.append(book.get_title())
        return books

    def check_out_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                return book.check_out()
```

```
        return False

    def return_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                return book.return_book()
        return False
```

Test your solution

```
# Test the Library and Book classes
library = Library("City Library")
assert library.name == "City Library", "Test failed: Incorrect library name"
assert len(library.books) == 0, "Test failed: Library should start empty"

# Create and add books
book1 = Book("1984", "George Orwell", "123-4567890123")
book2 = Book("To Kill a Mockingbird", "Harper Lee", "456-7890123456")

assert library.add_book(book1) == True, "Test failed: Book should be added successful
assert library.add_book(book2) == True, "Test failed: Second book should be added suc

# Test listing books
books_list = library.list_books()
assert "1984" in books_list, "Test failed: Book should be in the list"
assert "To Kill a Mockingbird" in books_list, "Test failed: Second book should be in

# Test checking out a book
result = library.check_out_book("123-4567890123")
assert result == True, "Test failed: Book should be checked out successfully"
```

Test your solution

```
# Test the Library and Book classes
library = Library("City Library")
assert library.name == "City Library", "Test failed: Incorrect library name"
assert len(library.books) == 0, "Test failed: Library should start empty"

# Create and add books
book1 = Book("1984", "George Orwell", "123-4567890123")
book2 = Book("To Kill a Mockingbird", "Harper Lee", "456-7890123456")

assert library.add_book(book1) == True, "Test failed: Book should be added successful
assert library.add_book(book2) == True, "Test failed: Second book should be added suc

# Test listing books
books_list = library.list_books()
assert "1984" in books_list, "Test failed: Book should be in the list"
assert "To Kill a Mockingbird" in books_list, "Test failed: Second book should be in

# Test checking out a book
result = library.check_out_book("123-4567890123")
assert result == True, "Test failed: Book should be checked out successfully"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Books and Ebooks

Exercise Description

Create a Python class called `EBook` that inherits from a base class `Book`. The `EBook` class will represent a digital version of a book with additional features related to digital media.

Base Class: `Book`

1. **Attributes**:

    - `title`: A string representing the title of the book.
    - `author`: A string representing the author of the book.
    - `isbn`: A string representing the unique ISBN number of the book.
    - `is_checked_out`: A boolean attribute indicating if the book is currently checked out (default is `False`).

2. **Methods**:

    - `__init__(self, title, author, isbn)`: Initializes the `title`, `author`, `isbn`, and sets `is_checked_out` to `False`.
    - `check_out(self)`: Marks the book as checked out by setting `is_checked_out` to `True` and returns `True`. If the book is already checked out, returns `False`.
    - `return_book(self)`: Marks the book as returned by setting `is_checked_out` to `False` and returns `True`. If the book is not checked out, returns `False`.
    - `get_info(self)`: Returns a string with the details of the book, including its title, author, ISBN, and availability status.

Derived Class: EBook

1. **Additional Attributes**:

    - `file_size_mb`: A float representing the size of the eBook file in megabytes.
    - `format_type`: A string representing the format of the eBook (e.g., "PDF", "EPUB", "MOBI").

2. **Methods**:

    - `__init__(self, title, author, isbn, file_size_mb, format_type)`: Initializes the `title`, `author`, `isbn`, and sets the `file_size_mb` and `format_type` attributes.
    - `get_info(self)`: Overrides the `get_info()` method of the base class to include the additional attributes related to the eBook (file size and format type).
    - `download(self)`: Prints a message indicating that the eBook is being downloaded and displays the file size and format.

Solution

Test your code

Your solution

```
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
```

```
        self.author = author
        self.isbn = isbn
        self.is_checked_out = False

    def check_out(self):
        if not self.is_checked_out:
            self.is_checked_out = True
            return True
        else:
            return False

    def return_book(self):
        if self.is_checked_out:
            self.is_checked_out = False
            return True
        else:
            return False

    def get_info(self):
        status = "Available" if not self.is_checked_out else "Checked out"
        return f"Title: {self.title}\nAuthor: {self.author}\nISBN: {self.isbn}\nStatu

# Derived class
class EBook(Book):
    def __init__(self, title, author, isbn, file_size_mb, format_type):
        super().__init__(title, author, isbn)
        self.file_size_mb = file_size_mb
        self.format_type = format_type

    def get_info(self):
        base_info = super().get_info()
```

```
        return f"{base_info}\nFile Size: {self.file_size_mb} MB\nFormat: {self.format

    def download(self):
        print(f"Downloading '{self.title}' ({self.format_type} format, {self.file_siz
```

Test your solution

```
# Test the EBook class (which inherits from Book)
ebook = EBook("The Great Gatsby", "F. Scott Fitzgerald", "987-6543210987", 1.2, "EPUB
assert ebook.title == "The Great Gatsby", "Test failed: Incorrect title"
assert ebook.author == "F. Scott Fitzgerald", "Test failed: Incorrect author"
assert ebook.isbn == "987-6543210987", "Test failed: Incorrect ISBN"
assert ebook.file_size_mb == 1.2, "Test failed: Incorrect file size"
assert ebook.format_type == "EPUB", "Test failed: Incorrect format type"
assert ebook.is_checked_out == False, "Test failed: eBook should be initially availab

# Test inheritance - check out functionality
result = ebook.check_out()
assert result == True, "Test failed: eBook should be checked out successfully"
assert ebook.is_checked_out == True, "Test failed: eBook should be marked as checked

# Test overridden get_info method
info = ebook.get_info()
assert "The Great Gatsby" in info, "Test failed: Info should contain title"
assert "1.2" in info, "Test failed: Info should contain file size"
assert "EPUB" in info, "Test failed: Info should contain format"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Timestamp

Exercise Description

In this exercise, you will create a `Timestamp` class to represent a moment in time. Inspired by the UNIX timestamp, it will count the number of seconds since the epoch (January 1, 1970, 00:00:00 UTC). To achieve this, you must implement time and date calculations manually, following these rules:

1. **Epoch Definition**: The epoch starts at January 1, 1970, 00:00:00 UTC.

2. **Calendar Rules**:

    - A year has 365 days, except for leap years, which have 366 days.
    - Leap years are years divisible by 4, except for years divisible by 100 but not divisible by 400.
    - Months have the following number of days:
        - January: 31, February: 28 (29 in leap years), March: 31, April: 30, May: 31, June: 30, July: 31, August: 31, September: 30, October: 31, November: 30, December: 31.

Your solution

```
class Timestamp:
    DAYS_IN_MONTH = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
    SECONDS_IN_MINUTE = 60
    SECONDS_IN_HOUR = 3600
    SECONDS_IN_DAY = 86400

    def __init__(self, seconds: int = 0):
        """Initialize the Timestamp with seconds since the epoch."""
        self.seconds = seconds

    @staticmethod
    def is_leap_year(year: int) -> bool:
        """Check if a given year is a leap year."""
```

```python
        return (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)

    @staticmethod
    def days_in_month(year: int, month: int) -> int:
        """Return the number of days in a given month of a specific year."""
        if month == 2 and Timestamp.is_leap_year(year):
            return 29
        return Timestamp.DAYS_IN_MONTH[month - 1]

    def to_date(self) -> tuple:
        """Convert the timestamp to (year, month, day, hour, minute, second)."""
        seconds = self.seconds
        year = 1970

        # Handle negative timestamps
        while seconds < 0:
            year -= 1
            seconds += 365 * self.SECONDS_IN_DAY + (self.SECONDS_IN_DAY if self.is_le

        # Calculate year
        while True:
            days_in_year = 365 + (1 if self.is_leap_year(year) else 0)
            if seconds < days_in_year * self.SECONDS_IN_DAY:
                break
            seconds -= days_in_year * self.SECONDS_IN_DAY
            year += 1

        # Calculate month
        month = 1
        while True:
            days_in_month = self.days_in_month(year, month)
```

```
            if seconds < days_in_month * self.SECONDS_IN_DAY:
                break
            seconds -= days_in_month * self.SECONDS_IN_DAY
            month += 1

        # Calculate day, hour, minute, second
        day = seconds // self.SECONDS_IN_DAY + 1
        seconds %= self.SECONDS_IN_DAY
        hour = seconds // self.SECONDS_IN_HOUR
        seconds %= self.SECONDS_IN_HOUR
        minute = seconds // self.SECONDS_IN_MINUTE
        second = seconds % self.SECONDS_IN_MINUTE

        return year, month, day, hour, minute, second

    @staticmethod
    def from_date(year: int, month: int, day: int, hour: int = 0, minute: int = 0, se
        """Create a timestamp from (year, month, day, hour, minute, second)."""
        total_seconds = 0

        # Add seconds for years
        for y in range(1970, year):
            total_seconds += 365 * Timestamp.SECONDS_IN_DAY + (Timestamp.SECONDS_IN_D

        # Subtract seconds for years before epoch
        for y in range(1970, year, -1):
            total_seconds -= 365 * Timestamp.SECONDS_IN_DAY + (Timestamp.SECONDS_IN_D

        # Add seconds for months
        for m in range(1, month):
```